JavaFX Bean Property与Binding对象

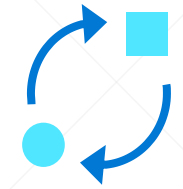
北京理工大学计算机学院
金旭亮

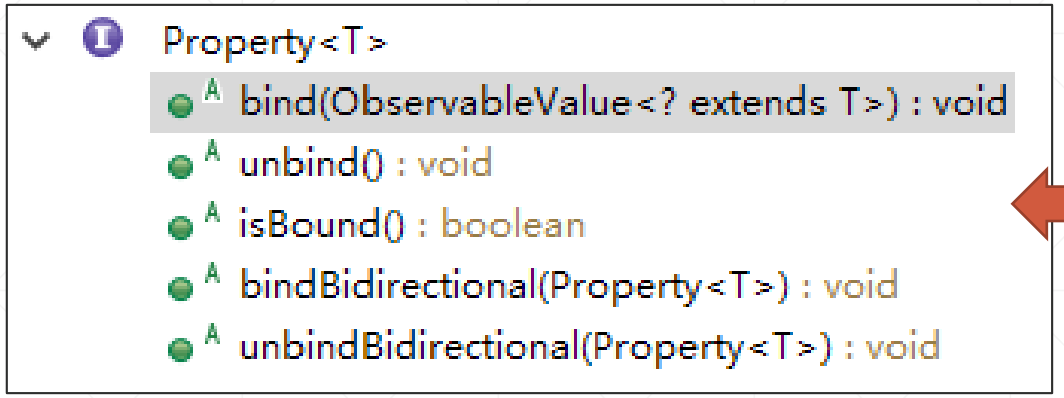
概述

- 前面课程介绍过，我们可以给JavaFX Bean Property附加监听器，从而当其值改变时，“自动地做某些事情”。

- 除了监听器，JavaFX提供了一些Binding对象，也能实现相同的目的，并且它的使用要比监听器简单。

两个相互绑定的JavaFX Property



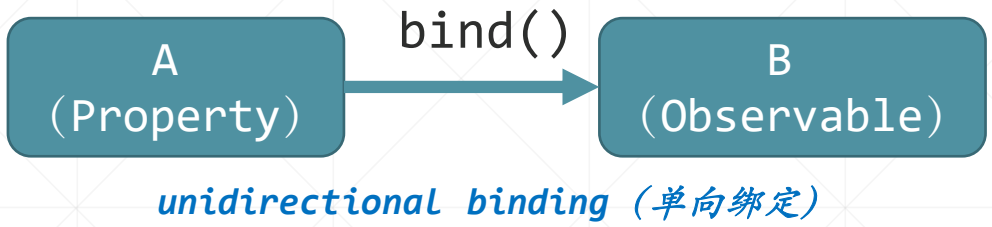


JavaFX Bean Property
实现了Property<T>接口

可以使用Property<T>接口中定义的**bind()**方法将两个JavaFX Bean Property “绑定”到一起，从而让两个属性值保持同步。

这个特性，正是数据绑定技术所需要的，只需要把绑定的一方设定为UI控件的属性，另一方设定为JavaBean的属性，就能很好地实现“**底层数据一变，UI界面立即刷新**”这一功能。

单向绑定与双向绑定



B.set(...), OK

A.set(...), **RuntimeException**

B.set(), OK

A.set(), OK

单向绑定示例

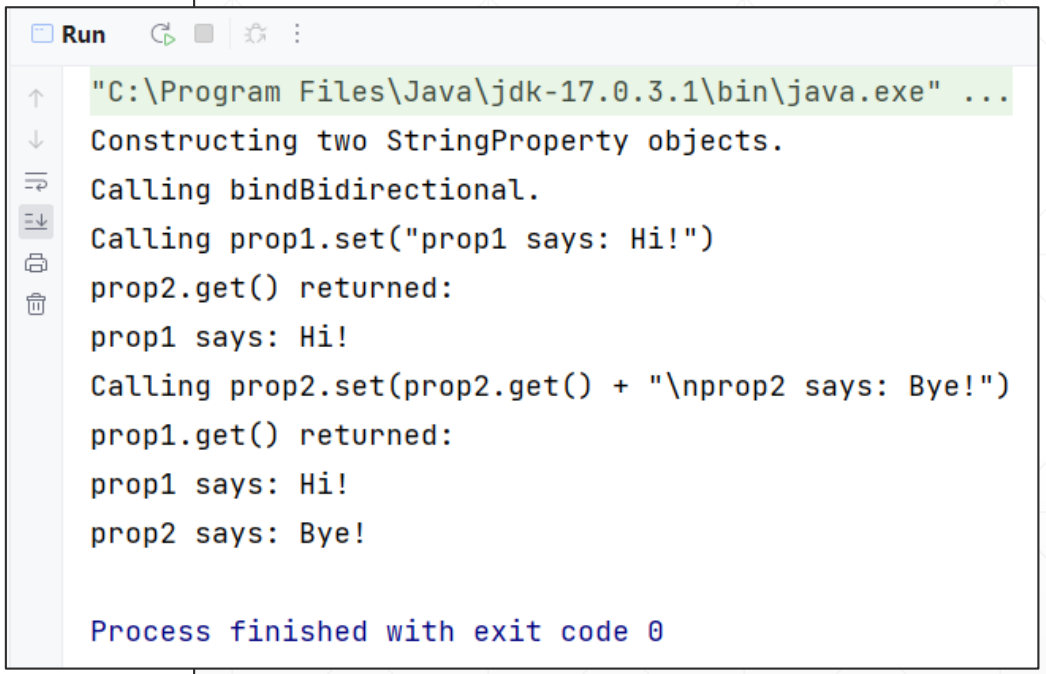
```
private static void bindAndUnbindOnePropertyToAnother() {
    //创建两个JavaFX Property
    IntegerProperty intProperty = new SimpleIntegerProperty( initialValue: 1024);
    IntegerProperty otherProperty = new SimpleIntegerProperty( initialValue: 0);
    System.out.println("intProperty.get()="+intProperty.get());
    System.out.println("otherProperty.get() = " + otherProperty.get());
    System.out.println("Binding otherProperty to intProperty.");

    //在两个JavaFX Property中建立绑定关系,绑定一建立，两者之间的值立即同步
    otherProperty.bind(intProperty);

    //赋值测试
    System.out.println("otherProperty.get() = " + otherProperty.get());
    System.out.println("Calling intProperty.set(7168).");
    intProperty.set(7168);
    System.out.println("otherProperty.get() = " + otherProperty.get());
    //单向绑定，绑定的一方自己不能设置值，以下这句将引发RuntimeException
    //otherProperty.set(999);
```

```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe"
intProperty.get()=1024
otherProperty.get() = 0
Binding otherProperty to intProperty.
otherProperty.get() = 1024
Calling intProperty.set(7168).
otherProperty.get() = 7168
Unbinding otherProperty from intProperty.
otherProperty.get() = 7168
Calling intProperty.set(8192).
otherProperty.get() = 7168

Process finished with exit code 0
```

```
    //解绑
    System.out.println("Unbinding otherProperty from intProperty.");
    otherProperty.unbind();

    //再次赋值测试
    System.out.println("otherProperty.get() = " + otherProperty.get());
    System.out.println("Calling intProperty.set(8192).");
    intProperty.set(8192);
    System.out.println("otherProperty.get() = " + otherProperty.get());
}
```

“双向绑定”示例

```
private static void bidirectionalBinding() {
    //创建两个JavaFX Property
    System.out.println("Constructing two StringProperty objects.");
    StringProperty prop1 = new SimpleStringProperty( initialValue: "");
    StringProperty prop2 = new SimpleStringProperty( initialValue: "");
    //建立双向绑定
    System.out.println("Calling bindBidirectional.");
    prop2.bindBidirectional(prop1);

    //在绑定双端都进行赋值测试
    System.out.println("Calling prop1.set(\"prop1 says: Hi!\")");
    prop1.set("prop1 says: Hi!");
    System.out.println("prop2.get() returned:");
    System.out.println(prop2.get());

    System.out.println("Calling prop2.set(prop2.get() + \"\\nprop2 says: Bye!\")");
    prop2.set(prop2.get() + "\nprop2 says: Bye!");
    System.out.println("prop1.get() returned:");
    System.out.println(prop1.get());
}
```



运行截图

“双向绑定”小结

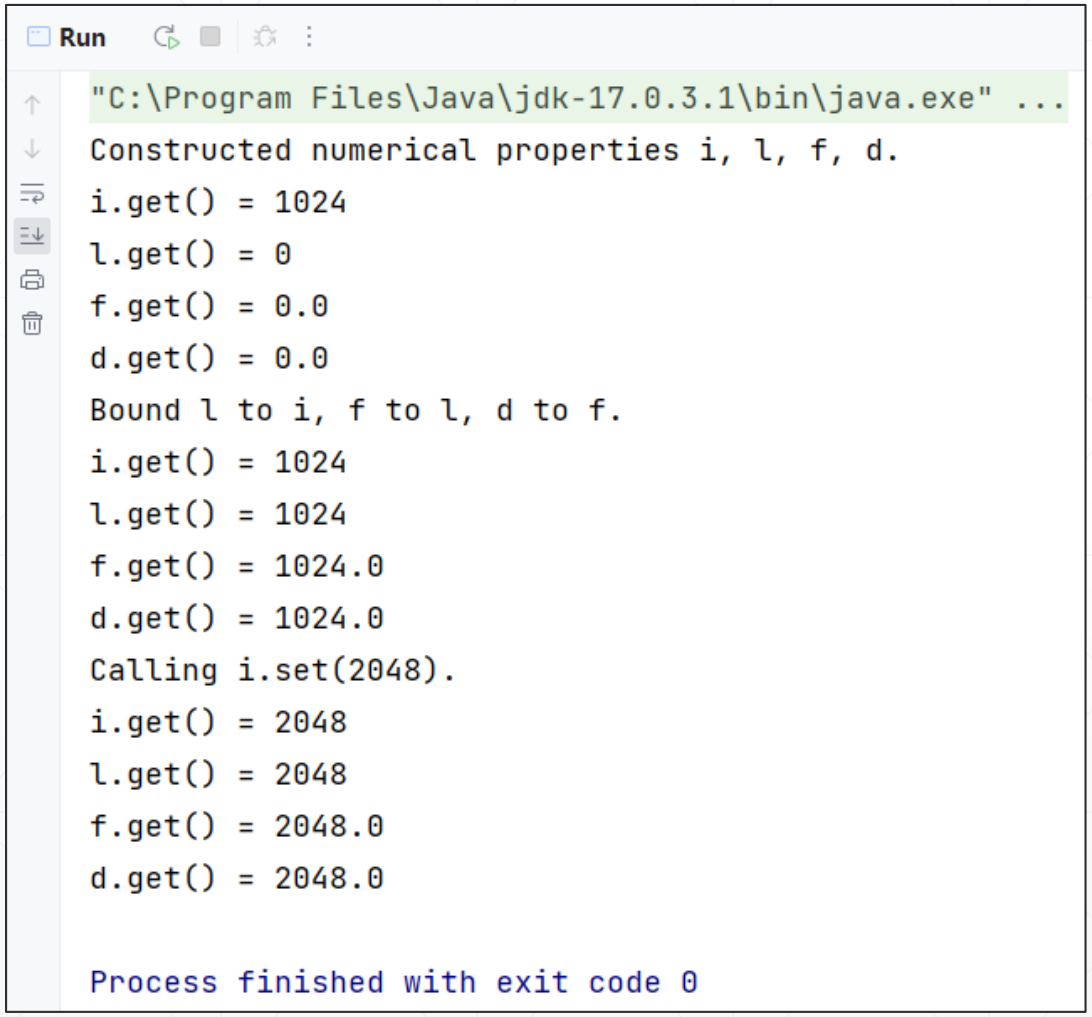
- 双向绑定只能建立在相同类型的JavaFX Bean Property中，它们的值，始终保持同步。
- 在JavaFX应用中，双向绑定多用于将JavaFX控件属性绑定到底层的数据源。

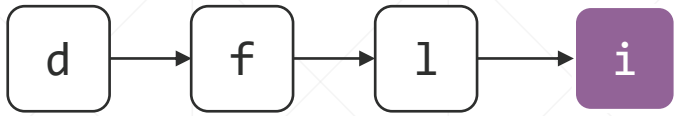
构建多级绑定

```
private static void bindingLink(){
    //创建四个JavaFX Bean Property对象
    IntegerProperty i = new SimpleIntegerProperty( bean: null,  name: "i",  initialValue: 1024);
    LongProperty l = new SimpleLongProperty( bean: null,  name: "l",  initialValue: 0L);
    FloatProperty f = new SimpleFloatProperty( bean: null,  name: "f",  initialValue: 0.0F);
    DoubleProperty d = new SimpleDoubleProperty( bean: null,  name: "d",  initialValue: 0.0);
    System.out.println("Constructed numerical properties i, l, f, d.");
    System.out.println("i.get() = " + i.get());
    System.out.println("l.get() = " + l.get());
    System.out.println("f.get() = " + f.get());
    System.out.println("d.get() = " + d.get());
    //基于四个JavaFX Bean Property对象构建单向绑定链
    l.bind(i);
    f.bind(l);
    d.bind(f);
    System.out.println("Bound l to i, f to l, d to f.");
    System.out.println("i.get() = " + i.get());
    System.out.println("l.get() = " + l.get());
    System.out.println("f.get() = " + f.get());
    System.out.println("d.get() = " + d.get());
    System.out.println("Calling i.set(2048).");
    i.set(2048);
    System.out.println("i.get() = " + i.get());
    System.out.println("l.get() = " + l.get());
    System.out.println("f.get() = " + f.get());
    System.out.println("d.get() = " + d.get());
    //单向绑定链，不允许在中间改变其值，以下这句引发异常：RuntimeException
    //f.set(999.0f);
}
```

只需要调用多次bind方法，就可以在多个JavaFX Bean Property之间建立多级绑定，“尾部”的值一更改，所有“上游”属性会全部得到更新。

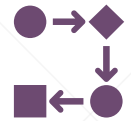
多级绑定运行结果





四个JavaFX Bean Property“串”在一起，对尾部对象的赋值，会传导到链表的前端。

流式绑定API



当有多个JavaFX Property需要协同，或者这些属性对象需要进行特定的处理（比如进行特定的计算），JavaFX提供了“Fluent Binding API”，可以采用级联调用的编程方式来完成这些工作。

```
//将double类型的属性，绑定到String类型的属性
text2.textProperty().bind(
  stackPane.rotateProperty().asString("%.1f"));
```

```
//进行简单的条件判断
text2.strokeProperty().bind(
    new When(rotate.statusProperty()
        .isEqualTo(Animation.Status.RUNNING))
        .then(Color.GREEN).otherwise(Color.RED)
);
```

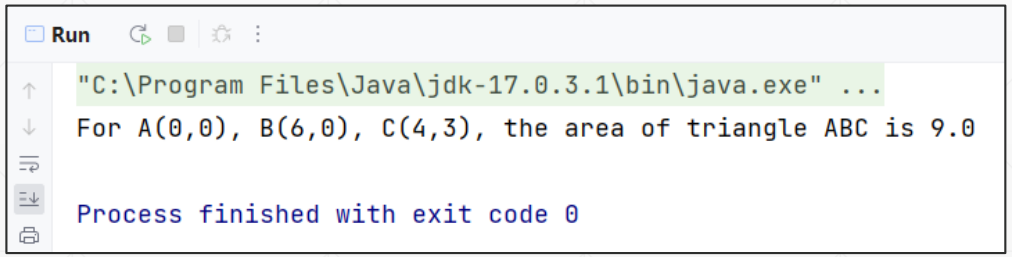
支持级联调用的这些方法，其返回值大都是某种类型的Binding对象或XXXBuilder对象，从而可以“串”在一起。

级联式方法调用

JavaFX的Binding对象非常有特色，我们重点看看它所支持的“级联方法调用”编程模式……

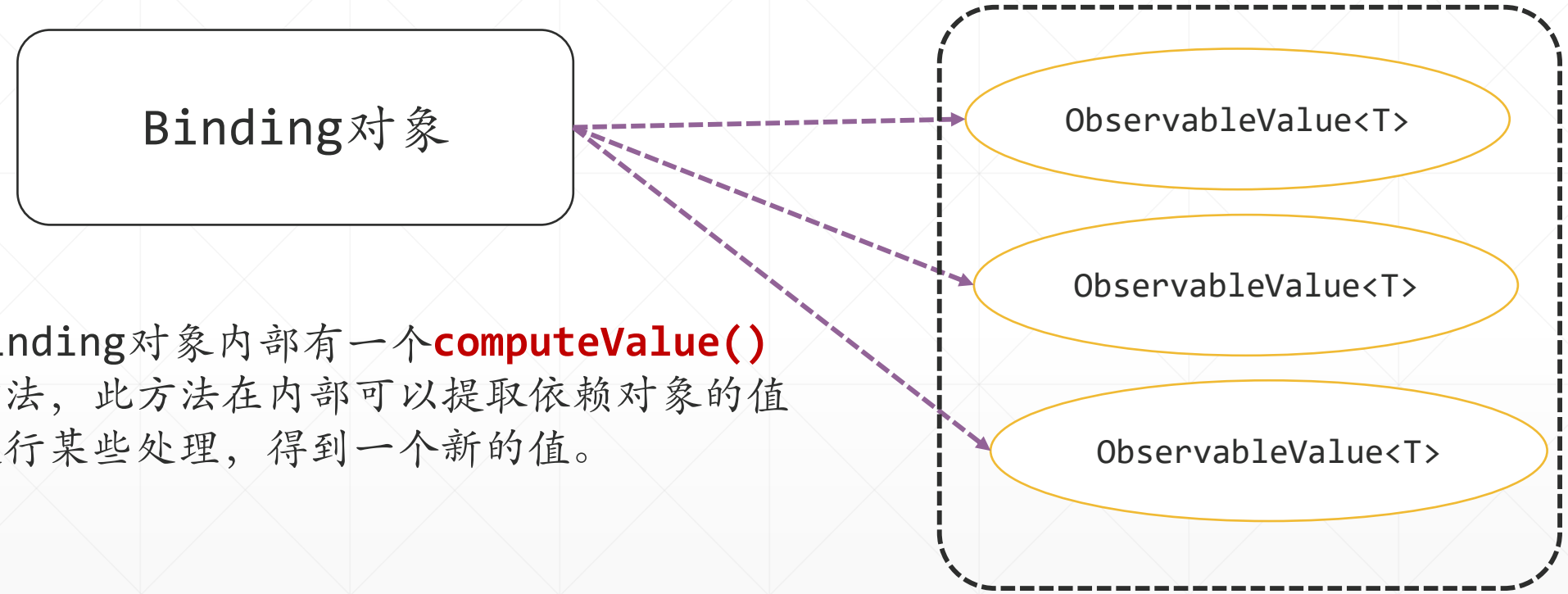
```
//创建6个IntegerProperty，代表三角形三个点的直角坐标
IntegerProperty x1 = new SimpleIntegerProperty( initialValue: 0);
IntegerProperty y1 = new SimpleIntegerProperty( initialValue: 0);
IntegerProperty x2 = new SimpleIntegerProperty( initialValue: 0);
IntegerProperty y2 = new SimpleIntegerProperty( initialValue: 0);
IntegerProperty x3 = new SimpleIntegerProperty( initialValue: 0);
IntegerProperty y3 = new SimpleIntegerProperty( initialValue: 0);

//构建三角形面积计算公式：S=1/2[(x1y2-x2y1)+(x2y3-x3y2)+(x3y1-x1y3)]
final NumberBinding area = x1.multiply(y2)
        .add(x2.multiply(y3))
        .add(x3.multiply(y1))
        .subtract(x1.multiply(y3))
        .subtract(x2.multiply(y1))
        .subtract(x3.multiply(y2))
        .divide( other: 2.0D);
```

```
//格式化输出
StringExpression output = Bindings.format(
        "For A(%d,%d), B(%d,%d), C(%d,%d), the area of triangle ABC is %3.1f",
        x1, y1, x2, y2, x3, y3, area);

//设置三点坐标值
x1.set(0); y1.set(0);
x2.set(6); y2.set(0);
x3.set(4); y3.set(3);
//输出结果
System.out.println(output.get());
```



理解Binding对象的作用

Binding对象可以有一到多个依赖对象，这些对象都实现了ObservableValue接口。



Binding对象内部有一个**computeValue()**方法，此方法在内部可以提取依赖对象的值进行某些处理，得到一个新的值。

JavaFX设计Binding对象，主要是考虑在实际开发中，程序员可以组合多个UI控件的特定属性进行某些处理，再将结果显示于屏幕上。

关于Binding对象……

- JavaFX的绑定对象，会在内部为它所依赖的所有数据对象附加监听器。
- 当依赖的任一数据对象值有变化时，此绑定对象居于“invalid”状态，这意味着它的值需要重新计算，才能获得真正有效的数值。

使用Binding对象构建支持条件的级联方法调用

其格式为：

```
new When(b).then(x).otherwise(y)
```

示例：已知三角形的三边长a,b,c,求三角形的面积

解：在a,b,c满足以下条件的前提下：

a + b > c, b + c > a, c + a > b.

三角形的面积可按以下公式计算：

Area = sqrt(s * (s - a) * (s - b) * (s - c))

其中，s=(a + b + c)/2

```java
//基于JavaFX的Binding对象，使用海伦公式计算三角形面积
1 usage
private static void useHeronsFormula() {
    DoubleProperty a = new SimpleDoubleProperty( initialValue: 0);
    DoubleProperty b = new SimpleDoubleProperty( initialValue: 0);
    DoubleProperty c = new SimpleDoubleProperty( initialValue: 0);
    //s=(a+b+c)/2
    DoubleBinding s = a.add(b).add(c).divide( other: 2.0D);
    //在满足“两边之和大于第三边”的前提下，用海伦公式计算三角形面积
    final DoubleBinding areaSquared = new When(a.add(b).greaterThan(c)
            .and(b.add(c).greaterThan(a))
            .and(c.add(a).greaterThan(b)))
            //满足条件开始计算
            .then(s.multiply(s.subtract(a))
                    .multiply(s.subtract(b))
                    .multiply(s.subtract(c)))
            .otherwise( otherwiseValue: 0.0D);
    //设定三边值
    a.set(3);
    b.set(4);
    c.set(5);
    System.out.printf("Given sides a = %1.0f, b = %1.0f, and c = %1.0f," +
                    " the area of the triangle is %3.2f\n", a.get(), b.get(), c.get(),
            Math.sqrt(areaSquared.get()));
}
```

程序输出：

```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...
Given sides a = 3, b = 4, and c = 5, the area of the triangle is 6.00

Process finished with exit code 0
```

```
private static void useHeronsFormula2() {
    final DoubleProperty a = new SimpleDoubleProperty( initialValue: 0);
    final DoubleProperty b = new SimpleDoubleProperty( initialValue: 0);
    final DoubleProperty c = new SimpleDoubleProperty( initialValue: 0);
    //重写基类的computeValue()方法，从此Binding对象的三个依赖对象中提取值进行计算
    DoubleBinding area = new DoubleBinding() {
        //使用初始化块，指明三个依赖对象
        {
            super.bind(a, b, c);
        }
        @Override
        protected double computeValue() {
            double a0 = a.get();
            double b0 = b.get();
            double c0 = c.get();
            //使用海伦公式计算三角形面积
            if ((a0 + b0 > c0) && (b0 + c0 > a0) && (c0 + a0 > b0)) {
                double s = (a0 + b0 + c0) / 2.0D;
                return Math.sqrt(s * (s - a0) * (s - b0) * (s - c0));
            } else {
                return 0.0D;
            }
        }
    };
```

除了使用JavaFX Binding类提供的方法，我们也可以自定义一个Binding类，重写它的computeValue()方法，来完成更灵活的处理工作。

```
    a.set(3);
    b.set(4);
    c.set(5);
    System.out.printf("Given sides a = %1.0f, b = %1.0f, and c = %1.0f," +
                      " the area of the triangle is %3.2f\n", a.get(), b.get(), c.get(),
            area.get());
}
```

运行截图

```
Run
"C:\Program Files\Java\jdk-17.0.3.1\bin\java.exe" ...
Given sides a = 3, b = 4, and c = 5, the area of the triangle is 6.00

Process finished with exit code 0
```

在JavaFX GUI应用中实现UI的自动更新

数值封装为MyNumber类中的一个名为number的JavaFX Bean Property，让Label控件的textPropery属性绑定到它。

点击按钮，更改number值，Label会自动地更新。

```
public class MyNumberController implements Initializable {
    2 usages
    @FXML
    private Label lblCounter;
    3 usages
    private final MyNumber myNumber = new MyNumber();

    1 usage
    @FXML
    protected void onHelloButtonClick() {
        myNumber.setNumber(myNumber.getNumber() + 1);
    }

    @Override
    public void initialize(URL url, ResourceBundle resourceBundle) {
        //建立单向绑定
        lblCounter.textProperty().bind(myNumber.numberProperty().asString());
    }
}
```

小结

本部分介绍了JavaFX Bean Property的基本特性与数据绑定的基础编程技巧，了解它们，有助于深入理解JavaFX的数据绑定机制并用好它们。

本部分内容无需死记，只需要认真地看看示例，读懂其源代码，并且弄清楚相关的接口和类的职责，会用SDK中提供的这些现成组件即可。